PW EARNERS

DSA in C++

DPP — 4 (SOLUTIONS)

Topic: Pattern Printing

---

*For Q1 to Q10: Analyze the nested loop architectures and predict the exact output **on your own; do not predict the output pattern by running the code**. If a question contains multiple choice options, select the correct one.*

Q1.

```cpp
#include<iostream>
using namespace std;
int main(){
    for(int i = 1; i <= 3; i++) {
        for(int j = 1; j <= 2; j++) {
            cout << i << j << " ";
        }
    }
}
```

Output:
11 12 21 22 31 32
Explanation: The outer loop runs 3 times, and for each iteration, the inner loop runs 2 times. The variables `i` and `j` are simply printed next to each other sequentially.

.....

Q2.

```cpp
#include<iostream>
using namespace std;
int main(){
    for(int i = 1; i <= 4; i++) {
        for(int j = 4; j >= i; j--) {
            cout << "*";
        }
        cout << endl;
    }
}
```

Which pattern is generated by this block?
Correct Option: A
Explanation: When $i = 1$, j runs from 4 down to 1 (4 stars). When $i = 2$, j runs from 4 down to 2 (3 stars). This creates a right-angled triangle pointing downwards.

.....

Q3.

```cpp
#include<iostream>
```

```cpp
using namespace std;
int main(){
    int n = 3;
    for(int i = 1; i <= n; i++) {
        char ch = 'A';
        for(int j = 1; j <= i; j++) {
            cout << ch++;
        }
        cout << endl;
    }
}
```

Output:

```
A
AB
ABC
```

Explanation: The character `ch` is reset to `'A'` at the start of every new row. The inner loop runs `i` times, printing and incrementing the character sequentially per row.

.....

Q4.

```cpp
#include<iostream>
using namespace std;
int main(){
    for(int i = 1; i <= 3; i++) {
        for(int j = 1; j <= i; j++) {
            if((i + j) % 2 == 0) cout << "1";
            else cout << "0";
        }
        cout << endl;
    }
}
```

Output:

```
1
01
101
```

Explanation: This uses coordinate addition to generate an alternating checkerboard pattern. If the sum of the row index `i` and column index `j` is even, it prints `1`. Otherwise, it prints `0`.

.....

Q5.

```cpp
#include<iostream>
using namespace std;
int main(){
    int count = 1;
    for(int i = 1; i <= 3; i++) {
        for(int j = 1; j <= 3; j++) {
            if(i == j) {
                cout << "X";
            } else {
                cout << count++;
```

```
            }
        }
        cout << endl;
    }
}
```

Output:
X12
3X4
56X
Explanation: When the row matches the column ($i == j$, the major diagonal), an `X` is printed. Otherwise, a continuously increasing global counter is printed.

.....

Q6.

```
#include<iostream>
using namespace std;
int main(){
    for(int i = 1; i <= 3; i++) {
        int val = i;
        for(int j = 1; j <= 3; j++) {
            cout << val << " ";
            val += 2;
        }
        cout << endl;
    }
}
```

Output:
1 3 5
2 4 6
3 5 7
Explanation: For each row i, the starting value is set to i. The inner loop iterates 3 times, printing the value and incrementing it by 2 each time.

.....

Q7.

```
#include<iostream>
using namespace std;
int main(){
    for(int i = 0; i < 4; i++) {
        for(int j = 0; j < 4; j++) {
            if(i == 0 || i == 3 || j == 0 || j == 3)
                cout << "#";
            else
                cout << " ";
        }
        cout << endl;
    }
}
```

What configuration does this coordinate filtering display?

Correct Option: C

Explanation: The `if` condition perfectly outlines the perimeter (first row, last row, first column, last column) of a 4x4 grid. The center is left blank with spaces, creating a hollow shell framework.

.....

Q8.

```cpp
#include<iostream>
using namespace std;
int main(){
    int n = 3;
    for(int i = 1; i <= n; i++) {
        for(int space = 1; space <= n - i; space++) cout << " ";
        for(int j = 1; j <= i; j++) cout << "*";
        cout << endl;
    }
}
```

Output:

```
  *
 **
***
```

Explanation: This is a standard right-aligned triangle. The first inner loop prints trailing spaces $(n - i)$, and the second loop prints the corresponding stars (i).

.....

Q9. ***(Crazy Question 1)***

```cpp
#include<iostream>
using namespace std;
int main(){
    int n = 4;
    for(int i = 1; i <= n; i++) {
        int x = 1;
        for(int j = 1; j <= n; j++) {
            if(j <= n - i) {
                cout << " ";
            } else {
                cout << x;
                x = !x;
            }
        }
        cout << endl;
    }
}
```

Output:

```
   1
  10
 101
1010
```

Explanation: Similar to a flipped triangle, but instead of stars, it prints alternating 1s and

0s. The variable `x` starts at 1 every row, and the boolean `!` operator logically flips it back and forth between 1 and 0.

.....

Q10. ***(Crazy Question 2)***

```cpp
#include<iostream>
using namespace std;
int main(){
    int k = 1;
    for(int i = 1; i <= 3; i++) {
        for(int j = 1; j <= i; i++) {
            cout << k++ << " ";
            if(k > 4) break;
        }
        if(k > 4) break;
        cout << endl;
    }
}
```

Output:
1 2 3 4
Explanation: This is a tricky execution flow. Notice the inner loop increments `i++` instead of `j++`. This means `j` stays 1 while `i` grows, creating what would normally be an infinite loop. However, `k` is continuously printing and incrementing. As soon as `k` hits 5, the `break` triggers, terminating both loops sequentially.

.....

For Q11 to Q20: Write clean, modular, and compilable C++ programs. Use proper nested loop structures. Assume user inputs an integer ***n*** *denoting structural constraints unless specified otherwise.*

Q11. Take an odd integer `n` as input from the user ($n \geq 3$). Write a program to print a **Plus inside Square Boundary** pattern.

```cpp
#include <iostream>
using namespace std;
int main() {
    int n;
    cin >> n;
    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= n; j++) {
            if(i == 1 || i == n || j == 1 || j == n || i == n / 2 +
               1 || j == n / 2 + 1)
                cout << "*";
            else
                cout << " ";
        }
        cout << endl;
    }
}
```

Explanation: We combine the boundary conditions (first row, last row, first column, last

column) with the plus intersection conditions (middle row and middle column). We isolate these coordinates to print stars while filling the rest of the grid with empty spaces.

.....

Q12. Take a positive integer `n` as input. Write a program to print a **Repeated Number Inverted Right Triangle**.

```cpp
#include <iostream>
using namespace std;
int main() {
    int n;
    cin >> n;
    for(int i = n; i >= 1; i--) {
        for(int j = 1; j <= i; j++) {
            cout << i << " ";
        }
        cout << endl;
    }
}
```

Explanation: We run the outer loop completely backward (from `n` down to 1). This value `i` natively acts as both the payload we print and the limiting constraint for the inner loop iterations.

.....

Q13. Write a program to print an **Alternating Row Offset Pattern** for a given integer `n` as shown below.

```cpp
#include <iostream>
using namespace std;
int main() {
    int n;
    cin >> n;
    for(int i = 1; i <= n; i++) {
        if (i % 2 == 0) {
            cout << " ";
        }
        int count = n - 1;
        if(i % 2 == 1) count++;
        for(int j = 1; j <= count; j++) {
            cout << i << " ";
        }
        cout << endl;
    }
}
```

Explanation: This pattern relies on checking if the current row index `i` is odd or even. If the row is even, a leading space is printed to offset the numbers, and the inner loop runs $n - 1$ times. If the row is odd, no leading space is printed, and the inner loop runs exactly n times.

.....

Q14. Write a program that prints an **Inscribed Hollow Diamond** pattern.

```cpp
#include <iostream>
using namespace std;
```

```cpp
int main() {
    int n;
    cin >> n;
    // Top Half
    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= n - i + 1; j++) cout << "*";
        for(int j = 1; j <= 2 * (i - 1); j++) cout << " ";
        for(int j = 1; j <= n - i + 1; j++) cout << "*";
        cout << endl;
    }
    // Bottom Half
    for(int i = n; i >= 1; i--) {
        for(int j = 1; j <= n - i + 1; j++) cout << "*";
        for(int j = 1; j <= 2 * (i - 1); j++) cout << " ";
        for(int j = 1; j <= n - i + 1; j++) cout << "*";
        cout << endl;
    }
}
```

Explanation: The pattern consists of a top half and a bottom half. For each row `i` (from 1 to `n`), the number of stars on each side decreases $(n - i + 1)$, while the spaces in the center increase $(2 \times (i - 1))$. The bottom half mirrors this exactly by iterating `i` backward.

.....

Q15. Write a program to print a symmetric **Cross Alphabet Pattern** for a given integer `n`.

```cpp
#include <iostream>
using namespace std;
int main() {
    int n;
    cin >> n;
    int totalRows = 2 * n - 1;
    for(int i = 1; i <= totalRows; i++) {
        for(int j = 1; j <= totalRows; j++) {
            if(i == j || i + j == totalRows + 1) {
                int charOffset = (i <= n) ? (i - 1) : (totalRows - i
                    );
                cout << (char)('A' + charOffset);
            } else {
                cout << " ";
            }
        }
        cout << endl;
    }
}
```

Explanation: To form the "X" shape, we print characters only on the major diagonal (`i == j`) and the minor diagonal (`i + j == totalRows + 1`) of a $(2n - 1) \times (2n - 1)$ grid. The character offset increases in the top half and symmetrically decreases in the bottom half. Empty spaces fill the rest of the coordinates.

.....

Q16. Create a program that takes an odd integer `n` and prints a **Square with Diagonals**.

```cpp
#include <iostream>
using namespace std;
int main() {
    int n;
    cin >> n;
    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= n; j++) {
            if(i == 1 || i == n || j == 1 || j == n || i == j || i +
                j == n + 1)
                cout << "*";
            else
                cout << " ";
        }
        cout << endl;
    }
}
```

Explanation: This problem uses 6 combined boundary rules: 'i == 1' (Top), 'i == n' (Bottom), 'j == 1' (Left), 'j == n' (Right), 'i == j' (Major Diagonal), and 'i + j == n + 1' (Minor Diagonal).

.....

Q17. Write a program to construct an **Alphabet Palindrome Pyramid**.

```cpp
#include <iostream>
using namespace std;
int main() {
    int n;
    cin >> n;
    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= n - i; j++) cout << " ";
        char ch = 'A';
        for(int j = 1; j <= i; j++) cout << ch++;
        ch -= 2;
        for(int j = 1; j < i; j++) cout << ch--;
        cout << endl;
    }
}
```

Explanation: We split the pyramid construction into three parts per row: 1) Initial spaces. 2) Ascending characters (from A to the peak). 3) Descending characters. We manually shift the ASCII index back by 2 (`ch -= 2`) right after the peak to avoid printing the peak twice and correctly walk backward down the alphabet.

.....

Q18. Design a program to generate a geometric **Hollow Star Diamond**.

```cpp
#include <iostream>
using namespace std;
int main() {
    int n;
    cin >> n;
    // Top Half
    for(int i = 1; i <= n; i++) {
```

```
        for(int j = 1; j <= n - i; j++) cout << " ";
        cout << "*";
        if(i > 1) {
            for(int j = 1; j <= 2 * i - 3; j++) cout << " ";
            cout << "*";
        }
        cout << endl;
    }
    // Bottom Half
    for(int i = n - 1; i >= 1; i--) {
        for(int j = 1; j <= n - i; j++) cout << " ";
        cout << "*";
        if(i > 1) {
            for(int j = 1; j <= 2 * i - 3; j++) cout << " ";
            cout << "*";
        }
        cout << endl;
    }
}
```

Explanation: A hollow diamond is symmetrical. We print the top half by placing one star, spacing based on $2i - 3$, and a closing star for rows beyond the first. We then loop entirely backward from $n - 1$ down to 1 to mirror the exact same logic upside down.

.....

Q19. Write a program to print the classic **Butterfly Pattern**.

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cin >> n;
    // Top Half
    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= i; j++) cout << "*";
        for(int j = 1; j <= 2 * (n - i); j++) cout << " ";
        for(int j = 1; j <= i; j++) cout << "*";
        cout << endl;
    }
    // Bottom Half
    for(int i = n; i >= 1; i--) {
        for(int j = 1; j <= i; j++) cout << "*";
        for(int j = 1; j <= 2 * (n - i); j++) cout << " ";
        for(int j = 1; j <= i; j++) cout << "*";
        cout << endl;
    }
}
```

Explanation: The butterfly pairs a left-aligned triangle with a right-aligned triangle, separated by calculated gaps. The gap formula is exactly $2 \times (n - i)$. The bottom half runs the same inner loops but simply steps 'i' backward to shrink the wings down.

.....

Q20. Write a program to print an **Expanding Number Diamond** pattern for a given integer

n.

```cpp
#include <iostream>
#include <cmath>
using namespace std;
int main() {
    int n;
    cin >> n;
    int totalRows = 2 * n - 1;
    for(int i = 1; i <= totalRows; i++) {
        int dist = abs(n - i);
        for(int j = 1; j <= dist; j++) {
            cout << "  ";
        }
        int startVal = n - dist;
        for(int j = 1; j <= n - dist; j++) {
            cout << startVal + j - 1 << " ";
        }
        for(int j = n - dist - 1; j >= 1; j--) {
            cout << startVal + j - 1 << " ";
        }
        cout << endl;
    }
}
```

Explanation: This problem requires managing a total of $2n - 1$ rows. Instead of splitting the code into a "Top Half" and "Bottom Half", we use the absolute difference from the center row (`abs(n - i)`) to calculate the exact number of spaces and elements needed for any given row. The row starts printing at the value `n - dist`, increments up to the center, and then decrements back down, mirroring itself perfectly.

.....

All the best! Keep Coding ♡